Database – No SQL – Mongodb

Session 5 –

1. **In a MongoDB collection named playlists (think Spotify), use the $set operator to update the 'genre' field to 'Chill' for the playlist with name 'Evening Vibes'.**

Ans:

1. Use $set to update the playlist genre

```
db.playlists.updateOne(
    { name: "Evening Vibes" },
    { $set: { genre: "Chill" } }
)
```

Verify:

```
db.playlists.findOne({ name: "Evening Vibes" })
```

2. **For a collection called restaurants (Zomato-style), remove the 'temporaryOffer' field from all documents where the offer has expired using the $unset operator.**

Ans:

Assuming expired offers are identified by an offerExpired: true field:

```
db.restaurants.updateMany(
    { offerExpired: true },
    { $unset: { temporaryOffer: "" } }
)
```

3. **Using the $push operator, add a new song 'Calm Waters' to the 'songs' array of the playlist document with _id: 101 in your playlists collection.

Hint: Use $push to add the song to the existing array without replacing it.**

Ans:

```
db.playlists.updateOne(
    { _id: 101 },
    { $push: { songs: "Calm Waters" } }
)
```

4. **In a Flipkart-like products collection, use the $addToSet operator to ensure the tag 'trending' is added only once to the 'tags' array for the product with SKU 'F12345', even if you run the update multiple times.**

Ans:

```
db.products.updateOne(
    { SKU: "F12345" },
    { $addToSet: { tags: "trending" } }
)
```

5. **Simulate a flash sale: in your products collection, use the $inc operator to decrease the 'stock' field by 1 for the product with name 'Wireless Earbuds' every time a purchase is made.

Constraint: Do not allow stock to go below zero.**

Ans:

To ensure stock never goes below zero, first match only products whose stock is greater than zero:

```
db.products.updateOne(
    {
        name: "Wireless Earbuds",
        stock: { $gt: 0 }
    },
    {
        $inc: { stock: -1 }
    }
)
```